

ДИСЦИПЛИНА	Фронтенд и бэкенд разработка
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям по дисциплине
ПРЕПОДАВАТЕЛЬ	Загородних Николай Анатольевич Краснослободцева Дарья Борисовна
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Практическое занятие 25

Инструменты сборки: Webpack и Vite

Рассмотрим современные инструменты сборки фронтенд-приложений — Webpack и Vite, их ключевые отличия, а также техники оптимизации бандла: code splitting, tree-shaking, lazy loading и анализ зависимостей. Решение практического задания осуществляется внутри соответствующей рабочей тетради, расположенной в СДО.

Что такое инструмент сборки

Современные фронтенд-приложения состоят из десятков и сотен файлов: JavaScript, TypeScript, CSS, изображения, шрифты. Браузер не умеет работать с модульной системой Node.js (`require` , `import/export`) напрямую в произвольном порядке, не знает о SASS, TypeScript и JSX. Инструмент сборки решает эти задачи:

- преобразует исходный код (TypeScript → JavaScript, SASS → CSS, JSX → JS);
- объединяет множество файлов в один или несколько оптимизированных бандлов;
- удаляет неиспользуемый код;
- минифицирует и оптимизирует для production;
- обеспечивает быструю обратную связь в режиме разработки (hot reload).

Два наиболее распространённых инструмента сборки — **Webpack** и **Vite**.

Webpack

Webpack — один из первых и наиболее зрелых бандлеров, появившийся в 2012 году. В основе его работы лежит построение **графа зависимостей**: Webpack начинает с точки входа (`entry`), обходит все `import / require` в коде и собирает единый граф всех модулей. Затем граф преобразуется в один или несколько выходных бандлов.

Ключевые понятия Webpack:

Entry — точка входа, с которой начинается построение графа зависимостей. Обычно это `src/index.js`.

Output — куда и под каким именем Webpack сохранит результат сборки.

Loaders — трансформаторы файлов. Webpack «из коробки» понимает только JavaScript и JSON. Loaders позволяют обрабатывать TypeScript, CSS, изображения и другие форматы.

Plugins — расширения, выполняющие задачи, выходящие за рамки трансформации файлов: генерация HTML, очистка папки `dist`, внедрение переменных окружения и т.д.

Mode — режим сборки: `development` (быстрая сборка, source maps) или `production` (минификация, tree-shaking).

Пример базовой конфигурации Webpack

```
// webpack.config.js
const path = require('path');
const HtmlWebpackPlugin = require('html-webpack-plugin');

module.exports = {
  entry: './src/index.js',
  output: {
    path: path.resolve(__dirname, 'dist'),
    filename: '[name].[contenthash].js',
    clean: true,
  },
  mode: 'production',
  module: {
    rules: [
      {
        test: /\.tsx?$/,
        use: 'ts-loader',
        exclude: /node_modules/,
      },
      {
        test: /\.css$/,
        use: ['style-loader', 'css-loader'],
      },
      {
        test: /\..(png|svg|jpg|jpeg)$/i,
        type: 'asset/resource',
      },
    ],
  },
  plugins: [
    new HtmlWebpackPlugin({ template: './public/index.html' }),
  ],
}
```

```
],
  resolve: {
    extensions: ['.tsx', '.ts', '.js'],
  },
};
```

Vite

Vite — инструмент сборки нового поколения, созданный Эваном Ю (автором Vue.js) в 2020 году. В отличие от Webpack, Vite использует принципиально иной подход:

- **В режиме разработки** Vite не собирает бандл совсем. Он запускает нативный ES-модульный сервер: браузер сам запрашивает нужные файлы по мере необходимости. Зависимости (node_modules) предварительно оптимизируются с помощью `esbuild` — крайне быстрого бандлера, написанного на Go.
- **В режиме production** Vite использует `Rollup` для сборки оптимизированного бандла.

Ключевые преимущества подхода Vite в разработке:

- Мгновенный холодный старт сервера — нет предварительной сборки всего проекта.
- Мгновенное HMR (Hot Module Replacement) — обновляется только изменённый модуль.

Пример конфигурации Vite

```
// vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  build: {
    outDir: 'dist',
    sourcemap: true,
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ['react', 'react-dom'],
        },
      },
    },
  },
  server: {
    port: 3000,
  },
});
```

```
    },  
  });
```

Сравнение Webpack и Vite

Характеристика	Webpack	Vite
Год выпуска	2012	2020
Dev-сервер	Собирает весь бандл при старте	Нативные ES-модули, без предсборки
Холодный старт	Медленный (секунды — минуты)	Мгновенный (< 1 секунды)
HMR	Пересобирает затронутые модули	Обновляет только изменённый файл
Production-сборка	Webpack + Terser	Rollup + esbuild
Конфигурация	Гибкая, но объёмная	Минималистичная, с разумными дефолтами
Экосистема и плагины	Очень зрелая, тысячи плагинов	Растёт, большинство Rollup-плагинов совместимы
Поддержка legacy-браузеров	Отличная (через Babel)	Требует дополнительной настройки (@vitejs/plugin-legacy)

Tip

Для новых проектов рекомендуется начинать с Vite — он значительно ускоряет разработку. Webpack остаётся актуальным для больших enterprise-проектов с нестандартными требованиями или существующей кодовой базой.

Code Splitting (разделение кода)

Code splitting — техника разбиения итогового бандла на несколько частей (чанков), которые загружаются браузером по мере необходимости, а не все сразу. Это уменьшает время до первой отрисовки страницы (Time to Interactive).

Существует два основных способа:

1. Динамический импорт

Стандартный синтаксис `import()` сигнализирует бандлеру, что модуль должен быть вынесен в отдельный чанк:

```
// Без code splitting - модуль всегда попадает в основной бандл  
import { heavyFunction } from './heavyModule';
```

```
// С code splitting – чанк загружается только при вызове
async function loadHeavyModule() {
  const { heavyFunction } = await import('./heavyModule');
  heavyFunction();
}
```

2. Lazy loading компонентов в React

В React динамический импорт сочетается с `React.lazy` и `Suspense` :

```
import React, { Suspense, lazy } from 'react';

// Компонент загружается только когда он впервые рендерится
const HeavyChart = lazy(() => import('./components/HeavyChart'));

function Dashboard() {
  return (
    <Suspense fallback=<div>Загрузка...</div>>
      <HeavyChart />
    </Suspense>
  );
}
```

При сборке `HeavyChart` и все его зависимости будут вынесены в отдельный файл `HeavyChart.[hash].js`, который браузер запросит только при первом рендере `Dashboard`.

Ручное разделение чанков в Webpack

```
// webpack.config.js
module.exports = {
  optimization: {
    splitChunks: {
      chunks: 'all',
      cacheGroups: {
        vendor: {
          test: /[\\/]node_modules[\\/]/,
          name: 'vendors',
          chunks: 'all',
        },
        react: {
          test: /[\\/]node_modules[\\/](react|react-dom)[\\/]/,
          name: 'react-vendor',
          chunks: 'all',
        },
      },
    },
  },
}
```

```
    },  
  };
```

Tree-shaking (устранение мёртвого кода)

Tree-shaking — автоматическое удаление из бандла кода, который импортируется, но нигде не используется. Термин происходит от метафоры: дерево зависимостей «встряхивается», и неиспользуемые «листья» опадают.

Tree-shaking работает только с ES-модулями (`import / export`), поскольку их структура статически анализируема на этапе сборки. CommonJS (`require`) не поддаётся tree-shaking.

```
// utils.js — экспортируем три функции  
export function add(a, b) { return a + b; }  
export function subtract(a, b) { return a - b; }  
export function multiply(a, b) { return a * b; }  
  
// main.js — используем только одну  
import { add } from './utils';  
console.log(add(2, 3));  
  
// В итоговый бандл попадёт только функция add.  
// subtract и multiply будут удалены при tree-shaking.
```

⚠ Warning

Некоторые библиотеки имеют «побочные эффекты» при импорте (например, полифиллы или CSS). Чтобы не удалять такие файлы, нужно явно указать это в `package.json`:

```
{ "sideEffects": ["*.css", "./src/polyfills.js"] }
```

Если побочных эффектов нет: `{ "sideEffects": false }` — это включает максимально агрессивный tree-shaking.

Анализ бандла

Визуальный анализ состава бандла помогает выявить тяжёлые зависимости и принять решение об оптимизации.

webpack-bundle-analyzer

```
npm install --save-dev webpack-bundle-analyzer
```

```
// webpack.config.js
const { BundleAnalyzerPlugin } = require('webpack-bundle-analyzer');

module.exports = {
  plugins: [
    new BundleAnalyzerPlugin({
      analyzerMode: 'static', // Генерирует HTML-отчёт
      openAnalyzer: true, // Автоматически открывает отчёт
      reportFilename: 'bundle-report.html',
    }),
  ],
};
```

После сборки откроется интерактивная карта бандла: каждый прямоугольник — это модуль, площадь соответствует его размеру.

rollup-plugin-visualizer (для Vite)

```
npm install --save-dev rollup-plugin-visualizer
```

```
// vite.config.js
import { visualizer } from 'rollup-plugin-visualizer';

export default defineConfig({
  plugins: [
    visualizer({
      filename: 'bundle-report.html',
      open: true,
      gzipSize: true,
    }),
  ],
});
```

Оптимизация production-сборки

Минификация

Webpack в режиме `production` автоматически применяет **Terser** для минификации JavaScript: удаляет пробелы, комментарии, укорачивает имена переменных.

Vite использует **esbuild** для минификации, который значительно быстрее Terser, хотя иногда уступает ему в степени сжатия.

Хэширование файлов

Добавление хэша содержимого к имени файла позволяет настроить агрессивное кэширование в браузере: файл с изменённым содержимым получит новое имя, и браузер не будет использовать устаревший кэш.

```
// webpack.config.js
output: {
  filename: '[name].[contenthash:8].js',
  chunkFilename: '[name].[contenthash:8].chunk.js',
}
```

Сжатие (Gzip / Brotli)

```
npm install --save-dev compression-webpack-plugin
```

```
const CompressionPlugin = require('compression-webpack-plugin');

module.exports = {
  plugins: [
    new CompressionPlugin({
      algorithm: 'brotliCompress',
      test: /\.js|css|html|svg$/,
      threshold: 10240, // Только файлы > 10 КБ
    }),
  ],
};
```

Анализ метрик: сравнение размеров бандла

Техника	До оптимизации	После оптимизации
Нет разделения	2.1 MB	—
+ Code splitting	—	450 KB (main)
+ Tree-shaking	—	320 KB
+ Минификация + Gzip	—	~95 KB

Практическое задание

Необходимо создать небольшое React-приложение с настроенным инструментом сборки и реализованными техниками оптимизации бандла.

В рамках выполнения задания требуется:

- создать React-приложение с использованием **Vite** (или Webpack по желанию);

- реализовать минимум **два маршрута** (например, главная страница и страница «О нас»);
- применить **lazy loading** для компонента на одном из маршрутов через `React.lazy` и `Suspense` ;
- добавить **анализатор бандла** (rollup-plugin-visualizer или webpack-bundle-analyzer) и приложить скриншот отчёта;
- убедиться, что production-сборка (`npm run build`) выполняется без ошибок.

Формат отчета

В качестве ответа на данный блок практик студентом подготавливается тематический проект. Критерии в [Практике 28](#)

Литература

1. [Документация Webpack](#)
2. [Документация Vite](#)